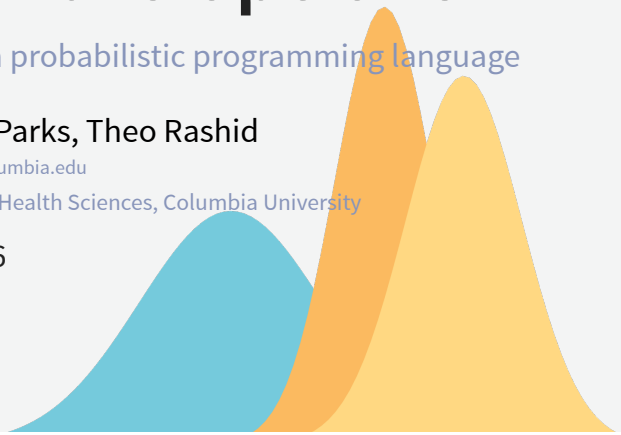
Software options

Choosing a probabilistic programming language

Robbie M. Parks, Theo Rashid
robbie.parks@columbia.edu
Environmental Health Sciences, Columbia University

Aug 6, 2026



Outline

- Why the software matters (and where it does not).
- A running example to translate.
- The sampling-based options.
- What goes wrong when you translate.
- A different paradigm: approximate inference.
- How to choose.
- Getting ready for the lab.

Why the software matters

The model and the software are different things

- So far we have written models in `NIMBLE`, but nothing we have done is *about* `NIMBLE`.
- The Poisson likelihood, the priors, the random effects: all of that is the **model**.
- The software is the machinery that turns that model into a posterior.
- A good habit: write the model down in math(s) first, then decide what to fit it with.

The model and the software are different things

- This matters practically:

- Reviewers ask about the model, not the package.
- Collaborators may use a different language than you do.
- The right tool for a pilot analysis is often not the right tool for the final one.
- Software changes far faster than statistics does.

What a probabilistic programming language does

A probabilistic programming language (PPL) lets you **declare** a model and hands the inference to an algorithm.

- You specify: the likelihood, the priors, the structure.
- It handles: gradients, samplers, tuning, and bookkeeping.
- You are spared writing a Metropolis-Hastings step by hand for every new model.

What you write, and what the PPL does

You supply	The PPL supplies
Likelihood	Sampler choice and tuning
Priors	Gradients (where needed)
Model structure	Convergence diagnostics
Data	Posterior draws or marginals

The boundary between these two columns is what really differs between languages.

Two families of inference

- **Sampling**: draw from the posterior, and use the draws to answer questions.
  - `BUGS`, `JAGS`, `NIMBLE`, `Stan`, `PyMC`, `numpyro`.
- **Approximation**: compute an approximation to the posterior directly, without drawing from it.
  - Variational inference, `INLA`.

A running example

The Italy provincial mortality model

- We will translate one model into several languages.
- It is the partial pooling model from the hierarchical modelling lab: monthly deaths by Italian province, with an expected count as offset and a province-level random effect.

The model, as maths

$$
\begin{aligned}
y_i &\sim \text{Poisson}(\mu_i) \quad i = 1, \ldots, N \\
\log(\mu_i) &= \log(E_i) + \alpha + \theta_{p[i]} \\
\theta_j &\sim \text{Normal}(0, \sigma_p) \quad j = 1, \ldots, N_p \\
\alpha &\sim \text{Normal}(0, 5) \\
\sigma_p &\sim \text{Half-Normal}(0, 1)
\end{aligned}
$$

Everything that follows is this model, written five different ways.

The model, in NIMBLE

```
nimbleCode({
  # priors
  alpha ~ dnorm(0, sd = 5)
  sigma_p ~ T(dnorm(0, sd = 1), 0, Inf)

  for (j in 1:Np) {
    theta[j] ~ dnorm(0, sd = sigma_p)
  }

  # likelihood
  for (i in 1:N) {
    y[i] ~ dpois(mu[i])
    log(mu[i]) <- log(E[i]) + alpha + theta[province[i]]
  }
})
```

What we are asking each language to do

- Accept a Poisson likelihood with an offset.
- Accept a vector of group-level effects with a shared, unknown standard deviation.
- Accept a half-normal prior on a positive parameter.
- Return the posterior for α, σ_p, and all 107 θ_j.

Every language below can do this. They differ in how much you have to write, and in what happens when it goes wrong.

The sampling-based options

BUGS: where all of this came from

- **B**ayesian inference **U**sing **G**ibbs **S**ampling, from the MRC Biostatistics Unit in Cambridge, late 1980s.
- The first widely used declarative model language: you write the model, it finds the sampler.
- `WinBUGS` and later `OpenBUGS` were the standard tools in epidemiology for two decades.
- Largely superseded now, but its syntax is the ancestor of everything in this section.

JAGS

- **J**ust **A**nother **G**ibbs **S**ampler (Plummer, 2003).
- A cross-platform reimplementation of the `BUGS` language, still actively used.
- Very stable, very well documented, and a lot of published epidemiological code is written in it.
- Weakness: Gibbs and slice sampling only, so it struggles with the correlated posteriors that arise in complex hierarchical models.

NIMBLE

- Takes the BUGS language and makes it programmable.
- Compiles your model to C++, so the sampling itself is fast.
- You can inspect and change the sampler assignment, and write your own distributions and samplers.
- This is why we started here: the code is readable, and nothing is hidden.

NIMBLE: what it is good at

- Good for:
  - Models where you want control over the sampler.
  - Custom distributions that no other package implements.
  - Anyone already fluent in BUGS or JAGS syntax.
- Less good for:
  - Large models, where the C++ compile step dominates the runtime.
  - Highly correlated posteriors, where the default samplers mix slowly.

Stan

- Developed at Columbia, and named after Stanislaw Ulam.
- Uses Hamiltonian Monte Carlo with the No-U-Turn Sampler, which we met on Day 1.
- Gradient-based, so it explores correlated posteriors far more efficiently than Gibbs.
- The documentation and the user community are the best in this space.

The model in Stan

```
data {
  int<lower=0> N;                        // number of observations
  int<lower=0> Np;                       // number of provinces
  array[N] int<lower=1, upper=Np> province;
  array[N] int<lower=0> y;               // deaths
  vector<lower=0>[N] E;                  // expected counts
}

parameters {
  real alpha;
  vector[Np] theta;
  real<lower=0> sigma_p;
}

model {
  alpha ~ normal(0, 5);
  sigma_p ~ normal(0, 1);                // half-normal via the lower bound above
  theta ~ normal(0, sigma_p);

  y ~ poisson_log(log(E) + alpha + theta[province]);
}
```

Stan: what it is good at

- Good for:
- Continuous parameters, especially in hard geometries.
- Anything where you want the best available diagnostics: divergences, tree depth, energy plots.
- Work that needs to be defensible to a statistically sophisticated reviewer.
- Less good for:
  - Discrete parameters, which have to be marginalised out by hand.
  - Quick exploratory work, because of the boilerplate you just saw.

Formula front-ends: brms and rstanarm

- Both write Stan code for you from an R formula.
- brms is remarkably general: multilevel, spatial, splines, distributional models, missing data.
- You can always ask it for the generated Stan code and take over by hand.
- For a large fraction of applied work, this is the fastest route to a correct model.

The model in brms

```r
library(brms)

fit <- brm(
  deaths ~ 1 + offset(log(expected)) + (1 | SIGLA),
  data = data,
  family = poisson(),
  prior = c(
    prior(normal(0, 5), class = "Intercept"),
    prior(normal(0, 1), class = "sd")
  )
)
```

The same model, in fewer lines.

Python: PyMC and numpyro

- PyMC is the most established Python option, and the closest in feel to Stan.
- numpyro is built on jax, the same backend as much modern deep learning.
- That means GPU support, and the ability to put a neural network inside a probabilistic model.
- Worth knowing if you collaborate with people outside statistics, where Python dominates.

The model in `numpyro`

```python
def model(Np, province, E, y=None):
    alpha = numpyro.sample("alpha", dist.Normal(0.0, 5.0))
    sigma_p = numpyro.sample("sigma_p", dist.HalfNormal(1.0))

    with numpyro.plate("provinces", Np):
        theta = numpyro.sample("theta", dist.Normal(0.0, sigma_p))

    with numpyro.plate("data", len(province)):
        log_rate = jnp.log(E) + alpha + theta[province]
        numpyro.sample("y", dist.Poisson(jnp.exp(log_rate)), obs=y)
```

Note how close this is in structure to the `NIMBLE` code, despite sharing no syntax at all.

Others you may meet

- `Turing.jl` — Julia, elegant, fast, small community.
- `TMB` and `RTMB` — Laplace approximation with automatic differentiation, popular in fisheries and ecology.
- `greta` — an `R` front-end on TensorFlow.
- `Pyro` — the PyTorch sibling of `numpyro`.

None of these are the wrong answer. The list is here so the names are not unfamiliar when you meet them.

Translating between languages

The syntax is the easy part

- The five versions above look different but describe the same object.
- Once you can read one, you can mostly read all of them.
- What actually causes trouble is not the syntax.

The most common translation bug

- Compare these two lines:

```
alpha ~ dnorm(0, 5)      # NIMBLE: precision, so sd = 1/sqrt(5) = 0.45
```

```
alpha ~ normal(0, 5);    // Stan: standard deviation, so sd = 5
```

These are not the same prior. One is eleven times tighter than the other.

NIMBLE inherits the BUGS convention of precision as the second argument. It accepts `sd =` and `var =` too, so always write which one you mean.

Parameterisations to check every time

Distribution	Watch out for
Normal	precision vs variance vs standard deviation
Gamma	rate vs scale
Negative binomial	several competing parameterisations
Beta	shape parameters vs mean and sample size
Half-normal	truncation vs a dedicated distribution

When a translated model gives different answers, this is the first thing to check.

Other things that do not translate

- **Indexing**: R and Stan count from 1, Python counts from 0.
- **Vectorisation**: writing `mu[1:n] <-` instead of a loop can change build time by an order of magnitude in NIMBLE, and does nothing in Stan.
- **Default priors**: brms and INLA supply them silently; NIMBLE and Stan do not.
- **Improper priors**: allowed in some languages, refused in others.

A different paradigm

Not all inference is sampling

- Everything so far draws from the posterior and uses the draws.
- The cost is time, and the price of stopping early is Monte Carlo error.
- But for some models we can compute the answer directly instead.
- Two approaches: variational inference, and INLA.

Variational inference, briefly

- Pick a simple family of distributions, then find the member closest to the posterior.
- Turns integration into optimisation, which is much faster.
- The approximation is often poor in the tails, and tends to understate uncertainty.
- Widely used in machine learning; used with care in epidemiology.

INLA, briefly

- **I**ntegrated **N**ested **L**aplace **A**pproximation (Rue, Martino & Chopin, 2009).
- Restricted to one model class: **latent Gaussian models**.
- Within that class, it computes the posterior **marginals** deterministically, using nested Laplace approximations.
- No chains, no burn-in, no rhat, and typically seconds rather than minutes.

The model in R-INLA

```r
formula <- deaths ~ 1 + f(SIGLA, model = "iid")

model <- inla(
  formula,
  data = data,
  E = expected,
  family = "poisson",
  control.predictor = list(link = 1),
  control.compute = list(config = TRUE)
)
```

Notice what is missing: no priors written out, no loop, no initial values, no chains. We unpack every one of those choices in the next session.

What you give up

- **Model class.** If your model is not a latent Gaussian model, `INLA` cannot fit it.
- **Joint posterior.** You get marginals. Functions of several parameters need extra work.
- **Transparency.** When `INLA` fails, the error message is often less informative than a bad traceplot.
- **A fixed approximation error** that does not shrink with more computation.

What you get

- Speed, by one to three orders of magnitude for the models we care about.
- Exact reproducibility: no seed, no Monte Carlo noise.
- No convergence diagnostics to check, because there is nothing to converge.
- Cheap sensitivity analyses, which makes the workflow from yesterday genuinely practical.

Coming next

- You saw the `MCMC` and `INLA` introduction on Day 1. The next session fills it in properly:
  - What a latent Gaussian model actually is, and how to tell whether yours is one.
  - How priors are specified in `INLA`, and why they are on the precision scale.
  - Spatial models, where `INLA` earns its reputation.
  - A direct comparison against `NIMBLE` on the same data.

How to choose

The honest answer

> Use whatever the people around you use, until you have a reason not to.

- The cost of a package is not the fitting time. It is the time you spend debugging alone.
- A slower tool with a colleague who knows it beats a faster tool with nobody to ask.

Four questions to ask

1. **Does my model fit the class?** If it is a latent Gaussian model, `INLA` is probably the fastest correct answer.
2. **How big is the problem?** Compile time and sampling time scale very differently across languages.
3. **Who maintains this after me?** A student, a collaborator, a journal's reproducibility checker.
4. **What does my field use?** Reviewers are more comfortable with familiar tools, fair or not.

Speed is not the only cost

Cost	Where it lands
Compile time	Paid once per model, dominates small problems
Sampling time	Scales with data and model complexity
Your time writing it	Highest in `Stan`, lowest in `brms` and `INLA`
Your time debugging it	Highest where the errors are least legible
Reviewer time	Lowest for whatever your field already knows

Summary

- The model is the thing. The software is a choice you make afterwards.
- Sampling languages differ mostly in ergonomics and sampler quality, not in what they can express.
- `INLA` is a genuinely different trade: a restricted model class in exchange for speed and determinism.
- Being able to read all of them, and translate between them, is more valuable than mastery of any one.

Questions?